

SISTEMA DE DETECCIÓN DE FRAUDE EN TIEMPO REAL

Documento de Arquitectura y Diseño Técnico (Versión AWS Learner Lab)

Arquitecto de Soluciones Cloud

Noviembre 2025

Índice

1. Resumen Ejecutivo	2
2. Arquitectura de Solución (Flujo de Datos)	2
2.1. Capa de Ingesta y Simulación	2
2.2. Capa de Procesamiento (The Brain)	2
2.3. Capa de Machine Learning (Económica)	2
2.4. Capa de Datos y Estado	3
2.5. Capa de Acción y Visualización	3
3. Estrategia de Implementación de Requerimientos Avanzados	3
3.1. A/B Testing (Champion/Challenger)	3
3.2. Explainability (SHAP Values)	3
3.3. Base de Datos de Grafos (Sin Neptune)	3
4. Estructura del Proyecto (Árbol de Carpetas)	4
5. Plan de Fases de Desarrollo	4
6. Gestión de Costos (Learner Lab)	5

1. Resumen Ejecutivo

El objetivo es desarrollar un sistema de detección de fraude financiero *end-to-end* capaz de procesar transacciones en tiempo real ($< 100\text{ms}$), utilizando Machine Learning para clasificación, lógica de grafos para detección de redes y flujos de trabajo automatizados para la investigación.

Restricción Principal: La arquitectura ha sido optimizada para ejecutarse dentro de un presupuesto de **\$100 USD** (AWS Learner Lab), sustituyendo servicios costosos de “infraestructura fija” (SageMaker Endpoints, Amazon Neptune) por arquitecturas “Serverless” y patrones de diseño eficientes (In-Memory Inference, DynamoDB Adjacency Lists).

2. Arquitectura de Solución (Flujo de Datos)

2.1. Capa de Ingesta y Simulación

- **Fuente:** Script Python local ejecutándose en bucle.
 - *Capacidades:* Generación de 1000+ TPS, inyección de anomalías (geo, monto) y simulación de *Concept Drift*.
- **Ingesta:** Amazon Kinesis Data Streams (1 Shard).
 - *Retención:* 24 horas (mínimo costo).

2.2. Capa de Procesamiento (The Brain)

- **Core Processor:** AWS Lambda (FraudRouter).
 - *Runtime:* Python 3.9 (con Layers personalizadas).
 - **Lógica:**
 1. **Feature Engineering:** Consulta rápida a DynamoDB para obtener perfil del usuario (Velocity Checks).
 2. **Graph Check:** Consulta de “Lista de Adyacencia” en DynamoDB para ver conexiones con estafadores.
 3. **Inferencia ML (A/B Testing):** Enrutamiento aleatorio (90/10) entre Modelo Champion y Challenger cargados en memoria.
 4. **Decisión:** Bloqueo o Aprobación.

2.3. Capa de Machine Learning (Económica)

- **Entrenamiento:** SageMaker Notebook (Instancia `ml.t3.medium`) o Local. **Se apaga inmediatamente tras el entrenamiento.**
- **Modelo:** XGBoost / Random Forest serializado (`.joblib`).
- **Despliegue:** El modelo se guarda en S3 y se carga dentro de la memoria de la función Lambda (evitando el costo de SageMaker Endpoints).
- **Explainability (SHAP):** Procesamiento asíncrono. Una segunda Lambda (`ExplainerFunction`) calcula los valores SHAP fuera de la ruta crítica de latencia.

2.4. Capa de Datos y Estado

- **Amazon DynamoDB (Modo On-Demand):**
 - **UserProfileTable:** Estado actual del usuario (última ubicación, contadores).
 - **UserGraphTable:** Modelo de grafo (quién ha compartido IP/Tarjeta con quién).
- **Amazon S3 (Data Lake):**
 - **raw-transactions/:** Histórico crudo (vía Firehose).
 - **ml-models/:** Artefactos de modelos.
 - **confirmed-fraud/:** Dataset para re-entrenamiento.

2.5. Capa de Acción y Visualización

- **Alertas:** Amazon SNS (Email/SMS a analistas).
- **Investigación:** Amazon SQS (Cola) → **AWS Step Functions** (Workflow de aprobación manual).
- **Analytics:** Amazon Athena (Consultas SQL) + Streamlit/QuickSight (Dashboard).

3. Estrategia de Implementación de Requerimientos Avanzados

3.1. A/B Testing (Champion/Challenger)

Implementado vía código dentro de la Lambda **FraudRouter**.

- Se cargan dos objetos de modelo: **model_v1** y **model_v2**.
- Un generador de números aleatorios decide qué modelo predice.
- El resultado se etiqueta en el log (**model_version: "v1"**) para análisis posterior en Athena.

3.2. Explainability (SHAP Values)

- **Desafío:** SHAP es lento (> 100ms).
- **Solución:** Desacoplamiento. Cuando se detecta fraude, el evento se envía a una segunda Lambda asíncrona. Esta calcula la contribución de cada variable y guarda el JSON resultante en S3/DynamoDB para visualización.

3.3. Base de Datos de Grafos (Sin Neptune)

- **Patrón:** Adjacency List Design Pattern en DynamoDB.
- **Estructura:** PK: **SourceID** (ej. IP_123), SK: **TargetID** (ej. User_A).
- **Uso:** Si **User_B** usa IP_123, la Lambda consulta la tabla. Si IP_123 está asociada a **User_A** (estafador), se marca la transacción por asociación.

4. Estructura del Proyecto (Árbol de Carpetas)

```

1 FraudDetectionSystem/
2 |-- 01_Data_Generator/
3 |   |-- transaction_generator.py           # Script principal de
      generacion
4 |   |-- drift_scenarios/                   # Configs para simular
      cambios
5 |
6 |-- 02_Lambda_Processor/
7 |   |-- lambda_function.py                # Logica Core (Router + ML)
8 |   |-- layer_content/                    # Librerias (pandas, xgboost
      )
9 |   |-- graph_logic/                      # Helpers para grafos en
      DynamoDB
10 |
11 |-- 03_ML_Model/
12 |   |-- training_notebook.ipynb          # Notebook de entrenamiento
13 |   |-- champion/                        # Artefacto del modelo V1
14 |   |-- challenger/                      # Artefacto del modelo V2
15 |
16 |-- 04_Infrastructure_Config/
17 |   |-- setup_commands.sh                 # Comandos CLI
18 |   |-- iam_policies.json                 # Permisos JSON
19 |
20 |-- 05_Investigation_Workflow/
21 |   |-- step_function_def.json            # Definicion del flujo (ASL)
22 |
23 |-- 06_Dashboard_Streamlit/
24 |   |-- app.py                            # Dashboard (Python)
25 |
26 |-- 07_Explainability/
27 |   |-- lambda_shap/                      #Codigo calculo asincrono
      SHAP

```

Listing 1: Estructura de Directorios del Proyecto

5. Plan de Fases de Desarrollo

1. **Fase 1: Cimientos.** Configuración de CLI local. Creación de Kinesis Stream, S3 Bucket y Tablas DynamoDB.
2. **Fase 2: Datos y Simulación.** Desarrollo del script generador de transacciones y verificación de flujo.
3. **Fase 3: Inteligencia (ML Local).** Entrenamiento de modelos XGBoost con dataset de Kaggle. Serialización y subida a S3.
4. **Fase 4: El Cerebro (Lambda Core).** Implementación de Lambda con Layers. Integración de lectura de Kinesis, DynamoDB y Predicción In-Memory.

5. **Fase 5: Funcionalidades Avanzadas.** Implementación de A/B Routing, cálculo de SHAP y Step Functions.
6. **Fase 6: Visualización.** Configuración de Athena y Dashboard.

6. Gestión de Costos (Learner Lab)

Para asegurar la supervivencia del saldo:

- **SageMaker:** Usar solo para Training Jobs. **Nunca** crear Endpoints.
- **DynamoDB:** Usar modo On-Demand.
- **Kinesis:** Limitar a 1 Shard.
- **Limpieza:** Detener el Lab siempre al finalizar la sesión.